



University of Piraeus

Faculty of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Security of Wireless and Mobile Networks

Android-Use Case 2

Supervising Professors: Christos Xenakis and Georgios Efthymoglou
(Grigorios Papoutsis)

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

18/06/2026

Abstract

This paper explores the vulnerabilities of authentication mechanisms in Android applications, analyzing three different login bypass methodologies. Through static analysis with JADX and APKTool (Smali Patching) and reverse engineering of native libraries with Ghidra, it is proven that hiding codes is not sufficient security. In addition, through dynamic analysis with the Frida framework, memory interference is achieved at runtime, bypassing any local control. The results confirm that client-side protections are precarious, highlighting the need for robust encryption and server-side verification.

Contents

Introduction.....	1
Practical piece of work.....	2
Bibliography	24

Introduction

The rapid evolution of mobile operating systems, dominated by Android, has transformed smartphones into essential tools for accessing sensitive personal and corporate data. At the same time, the complexity of applications has created a new challenge in the field of cybersecurity, the protection of code and authentication mechanisms that run on the user-side (client-side).

This paper focuses on the analysis and practical evaluation of the security mechanisms of an Android application, which attempts to shield its sensitive code (Flag) by transferring its logic to external native libraries (JNI-Java Native Interface). The aim of the research is not only to break the specific mechanism, but to investigate the effectiveness of different reverse engineering methodologies.

During the analysis, three distinct pillars of approach are examined:

1. **Static Analysis:** Through the reconstruction and modification of the Smali code, the vulnerability of static controls is highlighted.
2. **Reverse Engineering Native Code:** Using decompilation tools, the structure of the C/C++ library is analyzed to identify hidden paths.
3. **Dynamic Analysis:** Through the Frida framework, the manipulation of the application's memory at runtime is achieved, bypassing any static protection mechanism.

The paper aims to highlight that, in environments where code is executed on the user's device, "security through obscurity" techniques remain inadequate, underscoring the necessity for the adoption of robust server-side verification practices.

Practical piece of work

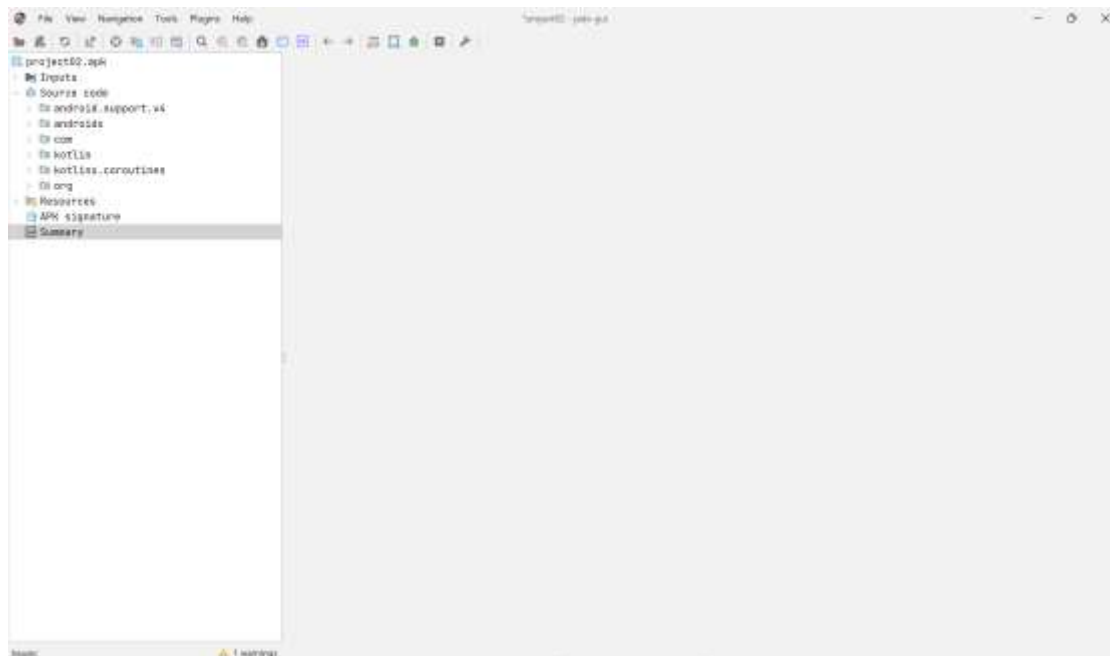
Method 1: JADX and APKTool (Smali Patching)

According to the requirements of the exercise (as defined in file README.txt), the first approach requires bypassing the login bypass exclusively by using the JADX

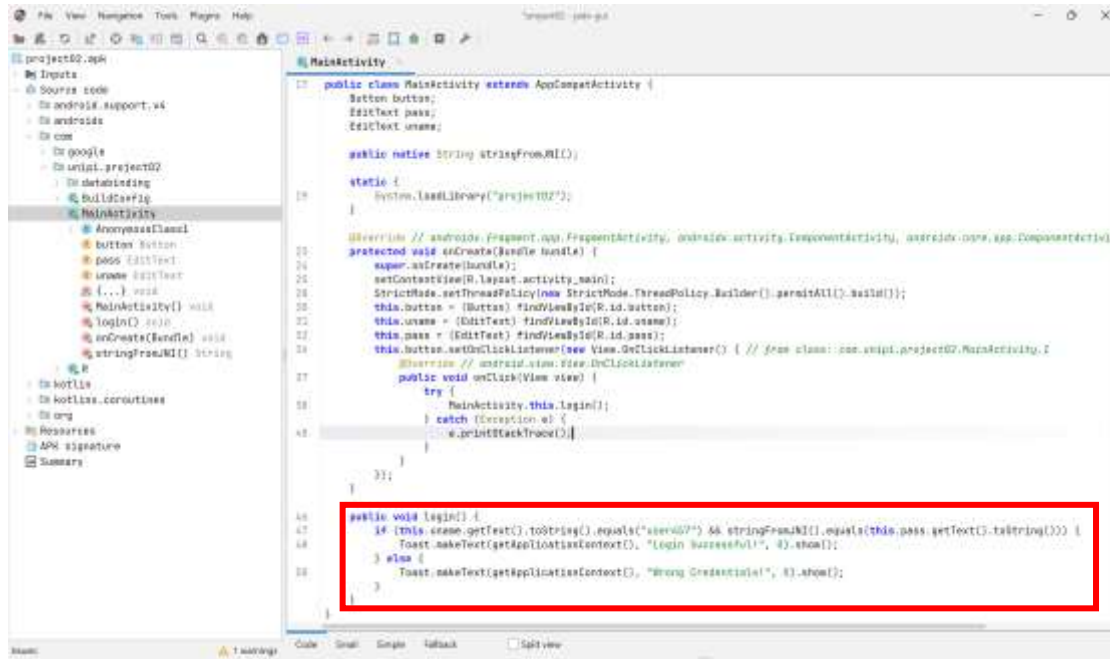
and APKTool tools. The main objective is to intervene in the execution logic of the application so that it is possible to successfully log in without the need to know or discover the real password.

In order to achieve this, the static analysis of the code was preceded to fully understand the architecture and control mechanisms implemented by the developer.

Unlike the Apktool tool—which disassembles the application by exporting the code in Smali format (an intermediate and illegible assembly-level representation language for the Dalvik engine)—the open-source JADX tool (JADX-GUI) functions as a decompiler. JADX undertakes the conversion of the executable file (APK/DEX) directly into Java source code, offering a clean and friendly environment for inspecting the application. Initially, the JADX-GUI program was executed on the local system.



The installation file was then project02.apk imported into the tool's environment in order to start the automatic process of decompressing and decompiling the code.



Through the navigation menu, the analysis focused on the source code catalog and specifically on the application package `com.unipi.project02`. There the main activity of the application named **MainActivity**.

By examining the generated Java code of the MainActivity class, the `login()` method was identified, which is responsible for checking the user's authentication when the login button is pressed. Through the analysis of the method, the logic of operation of the mechanism was fully revealed:

- **Username check:** The application retrieves the text entered by the user in the `uname` field and compares it (via the `.equals()`) command with the hardcoded string `"user457"`.
- **Password check:** At the same time, it retrieves the code from the pass field and compares it to the value returned by calling the native function `stringFromJNI()`.
- If the conditions are identical (if block), the app allows login by displaying the success message `"Login Successful!"` via a toast notification.
- If the comparison fails (else block), the message `"Wrong Credentials!"` is displayed and the input is rejected.

Considering that the goal of method 1 is to bypass the check without using the correct code (which is also dynamically generated by the JNI library), the proposed solution is to **Smali Patching**. This technique involves the immediate modification of the machine code (Smali) so that the comparison command is reversed. By

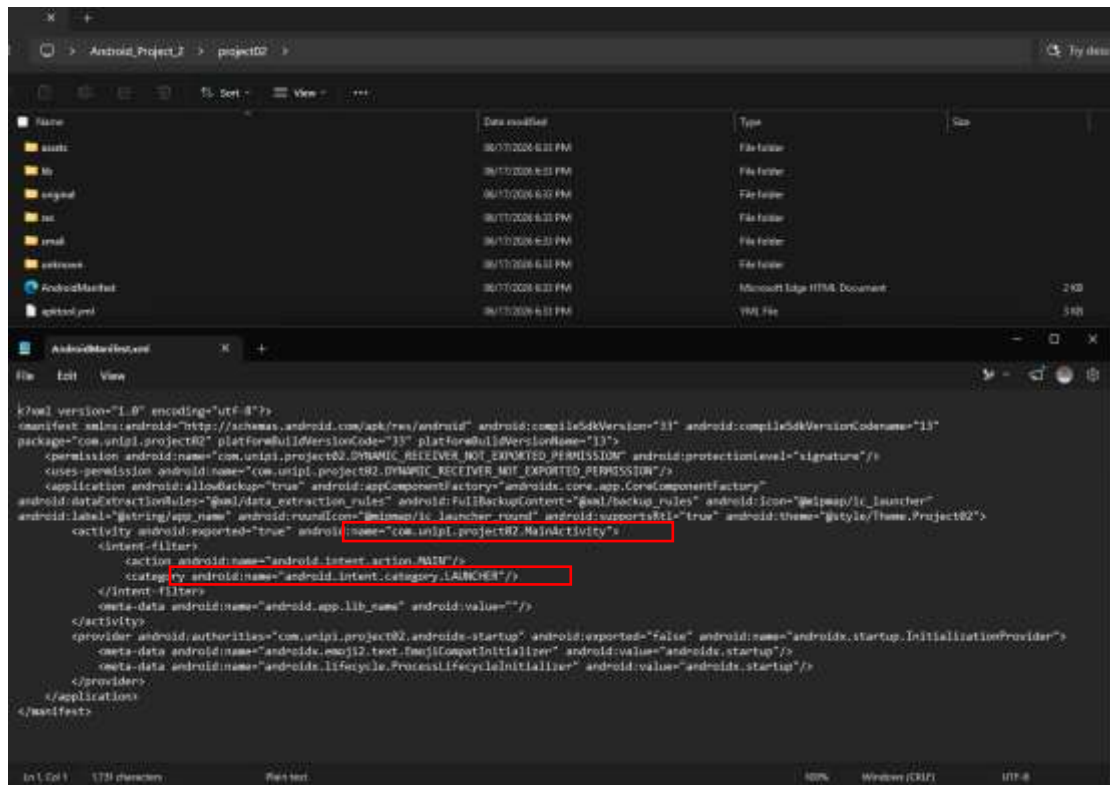
changing the logical condition of the function to consider the *incorrectly* elements, the complete circumvention of the mechanism is achieved.

In order to patch the code, it was necessary to extract the compressed files of the application. For this purpose, the open source tool was used **APKTool**, which allows the .apk executable to be disassembled and the code to be represented in Smali format.

The process was performed through the command line interface (Command Prompt), using the command **apktool d project02.apk**.

```
C:\Users\Kalai\Desktop\Android_Project_2>apktool d project02.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\Kalai\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...
C:\Users\Kalai\Desktop\Android_Project_2>
```

Upon completion of the command execution, a new local directory (project02) was created containing the source code (in Smali format), the resources and the basic configuration files of the application. Before directly searching for the control method in the application folders, the AndroidManifest.xml configuration file (which was opened with a simple word processor) was examined in order to map the structure of the application and identify the Main Activity.



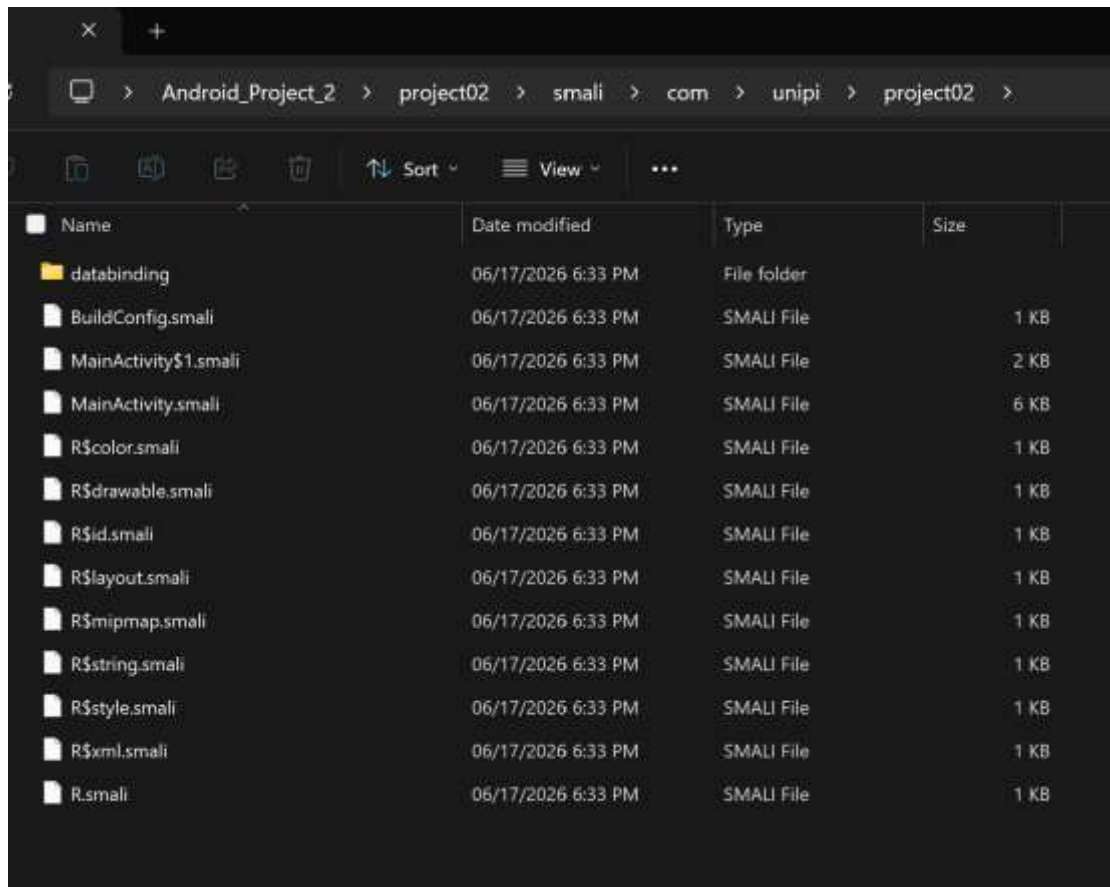
From the inspection of the XML code, two critical pieces of information were extracted:

1. The intent-filter containing the category "<category android:name='android.intent.category.LAUNCHER'>" confirmed that this activity is the home screen that loads when the application is launched (Login screen).
2. The statement 'android:name='com.unipi.project02.MainActivity'' revealed the exact path and file name containing the login screen code.

Utilizing this information, the navigation was targeted to the path project02/smali/com/unipi/project02/, bypassing irrelevant system directories and ready-made libraries (such as androidx or kotlin). Inside the project02 folder, the MainActivity.smali file was detected. This file was opened with a word processor and through a search, the authentication function that had already been observed in JADX, signed .method public login()V, was accurately identified.

Multiple files related to MainActivity (e.g., MainActivity\$1.smali) are found within the source code directory project02/smali/com/unipi/project02/. Files marked with the \$ sign represent anonymous sub-classes that are created during compilation, usually to handle events (such as button click listeners). Examination of these sub-

classes demonstrated that they simply redirect execution to the main login()V method. Therefore, the main focus of the analysis remained on the main MainActivity.smali file.



Inside MainActivity.smali, the execution flow of login()V is determined by two consecutive logical checks:

1. **Username check:** First, the application calls the equals method (which in Dalvik code translates as invoke-virtual) to compare the entered username with the default (hardcoded) value "user457". The result of this comparison is evaluated immediately after the conditional command if-eqz (If Equal to Zero). If the data do not match (the comparison returns 0/false), This command forcibly redirects the execution flow to the ':cond_0' tag, which leads to input being rejected and an error message appearing.
2. **Password check:** If the first check is successful, the program proceeds to check the code. Here, the exact same programming logic is followed (equals followed by if-eqz), except that the user's text is compared to the value produced and dynamically returned by the external (native) function stringFromJNI(). Similarly, if the codes do not match, the execution is diverted back to the failure label ": cond_0».

```

# virtual methods
.method public login()V
    .locals 3

    .line 47
    iget-object v0, p0, Lcom/example/project02/MainActivity;.>username:Landroid/widget/EditText;
    invoke-virtual {v0}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;
    move-result-object v0

    invoke-virtual {v0}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v0

    const-string v1, "user057"

    invoke-virtual {v0, v1}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    const/4 v1, 0x0

    if-eqz v0, :cond_0

    invoke-virtual {p0}, Lcom/example/project02/MainActivity;.>stringFromJNI()Ljava/lang/String;
    move-result-object v0

    iget-object v2, p0, Lcom/example/project02/MainActivity;.>password:Landroid/widget/EditText;
    invoke-virtual {v2}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;
    move-result-object v2

    invoke-virtual {v2}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v2

    invoke-virtual {v0, v2}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    if-eqz v0, :cond_0

    .line 48
    invoke-virtual {p0}, Lcom/example/project02/MainActivity;.>getApplicationContext()Landroid/content/Context;
    move-result-object v0

    const-string v1, "login Successful!"

```

To completely circumvent this mechanism, the technique of **Smali Patching**. This consists of replacing the "if-eqz" instructions with their logical opposite, the "if-nez" (If Not Equal to Zero) statements. With this functional code change, the importance of each check is reversed. Now each "if-nez" command diverts the flow to the failure tag (":cond_0") when the corresponding field matches the actual value, rather than when it differs from it. Therefore, the path to success ("Login Successful!") is only accessed when both fields (Username and Password) differ from the original credentials at the same time. Therefore, by deliberately entering incorrect values in both fields, entry is achieved without the need to know the real code, which, after modification, now

leads to rejection itself.

```
# virtual methods
.method public login()V
    .locals 3

    .line 47
    iget-object v0, p0, Lcom/unipi/project02/MainActivity;.>uname:Landroid/widget/EditText;

    invoke-virtual {v0}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;

    move-result-object v0

    invoke-virtual {v0}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v0

    const-string v1, "user457"

    invoke-virtual {v0, v1}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    const/4 v1, 0x0

    if-nez v0, :cond_0

    invoke-virtual {p0, Lcom/unipi/project02/MainActivity;.>getStringFromME()Ljava/lang/String;

    move-result-object v0

    iget-object v2, p0, Lcom/unipi/project02/MainActivity;.>pass:Landroid/widget/EditText;

    invoke-virtual {v2}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;

    move-result-object v2

    invoke-virtual {v2}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v2

    invoke-virtual {v0, v2}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    if-nez v0, :cond_0

    .line 48
    invoke-virtual {p0}, Lcom/unipi/project02/MainActivity;.>getApplicationContext()Landroid/content/Context;

    move-result-object v0

    const-string v2, "Login Successful"
```

By completing the modification of the Smali code, the next stage requires rebuilding the decompressed directory into a new, executable APK format file. This process was carried out through the terminal, executing the command **apktool b project02 -o project02_patched.apk**. The 'b' parameter denotes the build process, while the '-o' parameter specifies the name of the exported file (project02_patched.apk), ensuring that the original APK remains intact. The successful execution of the command was confirmed by the absence of errors and the creation of the new file in the working directory.

```
C:\Users\Kai\Desktop\Android_Project_2>apktool b project02 -o project02_patched.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Smaling smali folder into classes.dex...
I: AndroidManifest.xml and resources have not changed.
I: Building apk file...
I: Importing assets...
I: Importing lib...
I: Importing unknown files...
I: Built apk into: project02_patched.apk
C:\Users\Kai\Desktop\Android_Project_2>
```

project02	06/17/2026 8:30 PM	File folder
Whiteup_Template	06/12/2026 12:30 AM	File folder
apktool	06/17/2026 8:30 PM	Windows Batch File
apktool	06/17/2026 8:30 PM	Executable Jar File
project02.apk	06/12/2026 12:30 AM	APK File
project02_patched.apk	06/17/2026 8:33 PM	APK File
README	06/12/2026 12:30 AM	Text Document

Despite the successful rebuild, directly installing the project02_patched.apk file in the Android Studio environment becomes impossible. The Android operating system incorporates strict security mechanisms, explicitly prohibiting the installation of applications (packages) that do not bear a valid digital signature. Since Apktool reconstructs the file without signing it, it required the use of an external signing tool.

For this purpose, the standalone tool was used **uber-apk-signer**. Through the execution of the command **java -jar uber-apk-signer-1.3.0.jar -a project02_patched.apk**, the tool took care of the automatic alignment (zipalign) and the digital signature of the file. The result of the process was the production of the final, functional and digitally signed package called project02_patched-aligned-debugSigned.apk.

```
C:\Users\Walai\Desktop\Android_Project_2>java -jar uber-apk-signer-1.3.0.jar -a project02_patched.apk
Source: C:\Users\Walai\Desktop\Android_Project_2
Binary-lib/windows-x86_2\libwinpthread-1.dll
C:\Users\Walai\AppData\Local\Temp\usapksigner-7386785299193876389
zipalign location: BUILD_TN
C:\Users\Walai\AppData\Local\Temp\usapksigner-7386785299193876389\win-zipalign_33_4_1.exe16418115412522769127.tmp
Keystore: [0] e0a093b2 C:\Users\Walai\.android\debug.keystore (DEBUG_ANDROID_FOLDER)

#1. project02_patched.apk

SIGN
file: C:\Users\Walai\Desktop\Android_Project_2\project02_patched.apk (4.78 MiB)
checksum: 11198b625cfa0dd124a48d951d7e5e2f38dab386d465be9925713a0973e0eb (sha256)
- zipalign success
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.conscrypt.NativeLibrary$Util in an unnamed module (file:/C:/Users/Walai/Desktop/Android_Project_2/uber-apk-signer-1.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled
- sign success

VERIFY
file: C:\Users\Walai\Desktop\Android_Project_2\project02_patched-aligned-debugSigned.apk (4.86 MiB)
checksum: d69d9810fd28e61cc2f3f1f47a5299a893fe76f226c1a7c9c6857d37736148b (sha256)
- zipalign verified
- signature verified [v1, v2, v3]
42 warnings
Subject: C=US, O=Android, CN=Android Debug
SHA256: w962786f20f12b20a253af31126c3daach567c65b6f635ba5d2ccd88ff7ea / SHA256withRSA
Expires: Sat Apr 22 11:44:24 EEST 2056

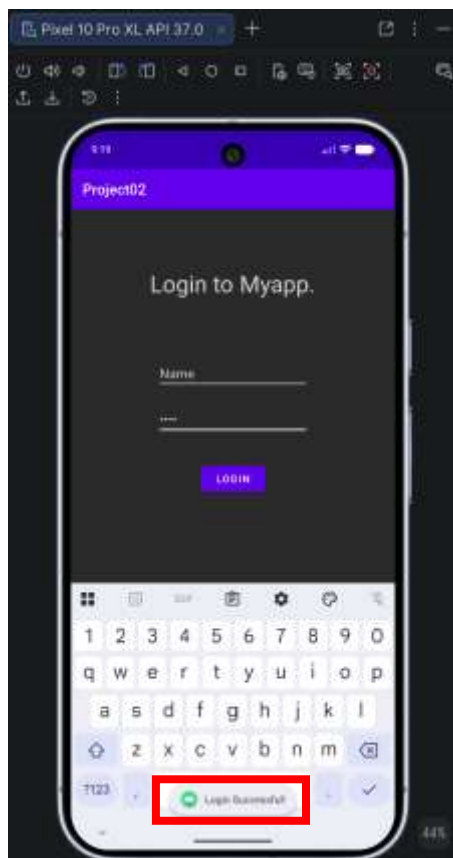
[Wed Jun 17 20:50:13 EEST 2026][v1.3.0]
Successfully processed 1 APKs and 0 errors in 1.11 seconds.
C:\Users\Walai\Desktop\Android_Project_2>
```

project02
Whiteup_Template
apktool
apktool
project02.apk
project02_patched.apk
project02_patched-aligned-debugSigned.apk
project02_patched-aligned-debugSigned.apk.idsig
README
uber-apk-signer-1.3.0

The final signed file project02_patched-aligned-debugSigned.apk was installed on the virtual device (Android Emulator) through a drag and drop process. During the execution of the application, an authentication test was performed by intentionally entering incorrect and random information (e.g., a space in the Username field and

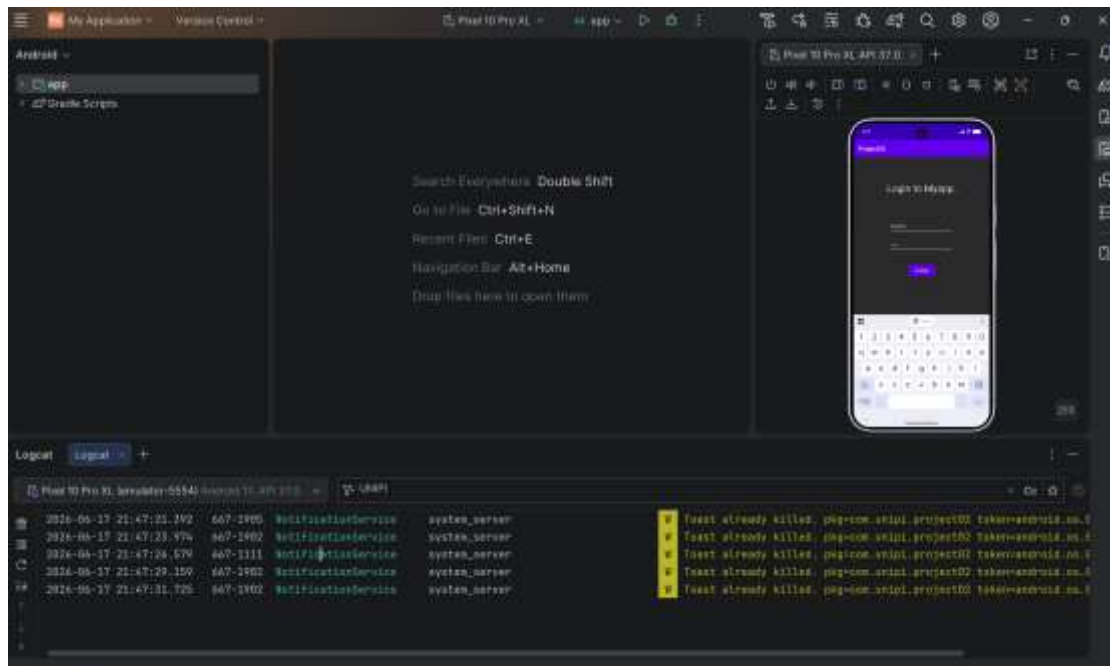
"1234" in the Password field). Upon submission of the details, the app evaluated the credentials.

Due to the previous reversal of logical controls (if-eqz to if-nez) within the Smali code, the importance of comparisons was reversed, with the result that the approval of input now requires elements that differ from the actual ones in both fields. For this reason, the test was carried out with an empty Username field and the random code "1234" - values that are both different from the original credentials - leading to the display of the message "Login Successful!" and confirming the bypass without using the real code.



During the successful execution of the bypass, it was observed that the application does not display the requested flag (in the form of `UNIPi{...}`) on the device screen, only the success message. In order to investigate the possibility that the flag was printed in the background during the identification process, the system logs were analyzed through the Logcat tab of Android Studio.

Applying a search filter with the keyword 'UNIPi', the only findings concerned system alerts (e.g., 'Toast already killed'), which were recorded solely because of the app's package name (com.unipi.project02).



As confirmed by the application's Java source code (in JADX), the mechanism simply prints the notification message (Toast) without handling the flag text. Therefore, method 1 was completed with complete success, as the primary goal of full Login Bypass without the knowledge of the password was achieved.

Method 2: JADX and Ghidra (Extracting Credentials via Reverse Engineering)

To start the second method, which aims to actually recover the hidden password, it was necessary to reset the control environment. The patched application was uninstalled from the virtual device, and the original, unmodified project02.apk file was reinstalled in its place.

During the initial static analysis with the JADX tool, the statement "System.loadLibrary("project02")?", as well as the call of the external function "stringFromJNI()" were detected within the MainActivity class. This finding indicates that the developer, for reasons of increased security against Dalvik-level disassembly, implemented the code verification mechanism in an external, native C/C++ (Shared Object) library. The actual password (Flag) is hardcoded inside. In order to retrieve the library in question, project02.apk file was treated as a compressed .zip format folder and opened using decompression software (7-Zip). Navigating to the lib/x86_64/ path (a directory containing the compiled libraries for the virtual processor architecture), the libproject02.so file was found.



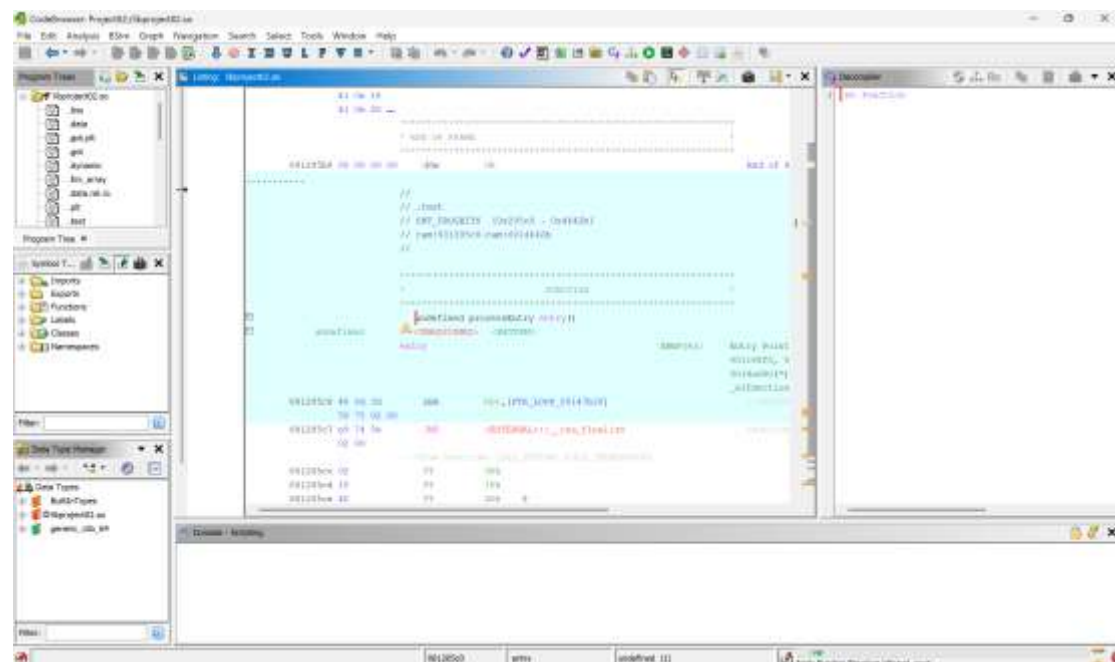
This file was exported to the local working directory, becoming the main target of the analysis. For the analysis of the library's compiled machine code, the specialized reverse engineering tool was used **Ghidra**. The libproject02.so file was imported into a new Project of the tool.

Upon import, the system successfully identified the file structure as a 64-bit ELF (Executable and Linkable Format), optimized for x86 architecture, completing the process without errors, as confirmed by the Import Results Summary.



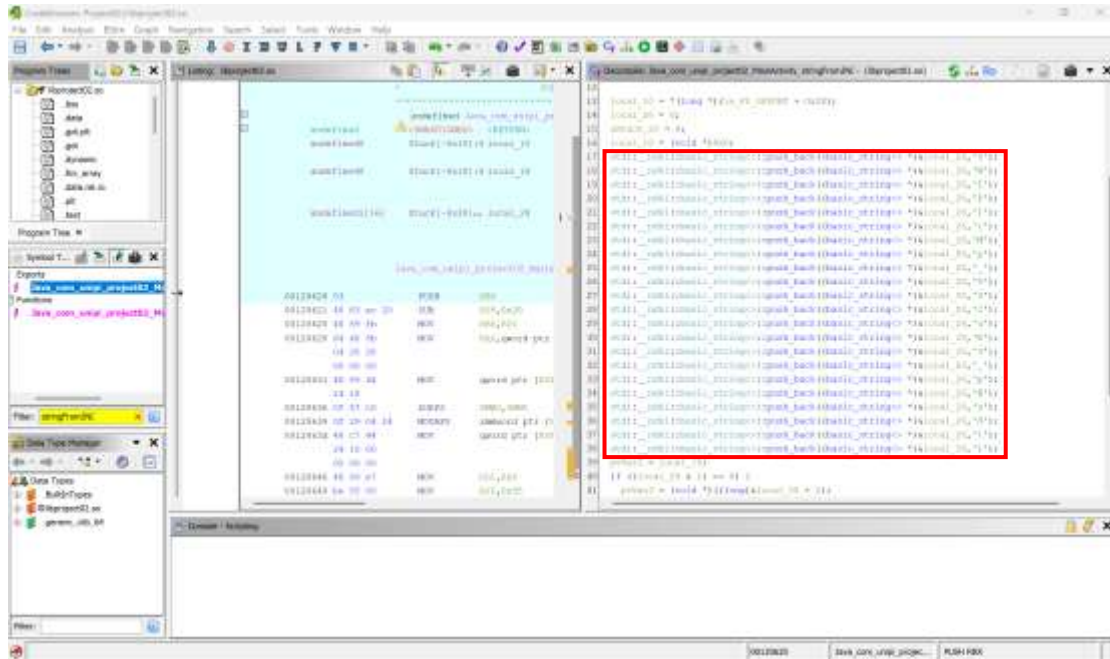
By opening the libproject02.so file in the main Ghidra interface, the process of automatic parsing was performed while maintaining the default settings of the tool in

order to map the functions and data structures of the library.



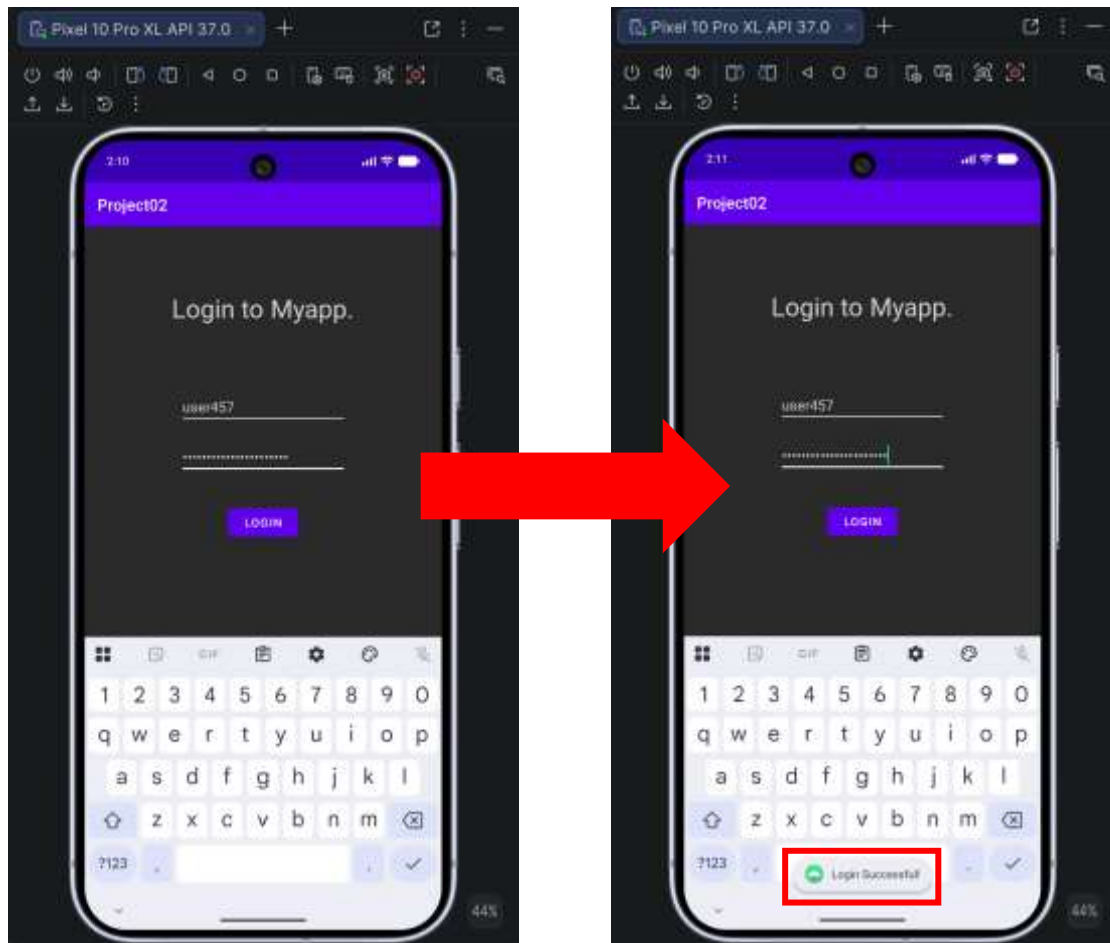
Based on the findings of static analysis from JADX (method 1), it was known that the application calls the native method `stringFromJNI()`. By using the Symbol Tree window and applying the corresponding search filter to the functions, the function in question was detected, which at compilation had received the full signature "`Java_com_unipi_project02_MainActivity_stringFromJNI`". By selecting the target function, the tool proceeded to debug by translating the machine code into a readable C/C++ format in the corresponding window.

From the examination of the source code, it was found that the creator of the application applied masking techniques to protect the code. In particular, the stack strings technique was used, where the desired alphanumeric is not stored as a single, continuous variable (which would be easily detectable with tools such as strings), but is dynamically constructed in memory at runtime. This is achieved through consecutive calls of the method `push_back` of the `std::__ndk1::basic_string` class, by entering the character-by-character code.



By unifying the individual characters in their order of execution in the code, Ghidra's tool completely deciphered the sequence, revealing the actual credential and at the same time the requested flag of the exercise: **UNIPI{My_53cuR3_p@sS!}**

To definitively verify method 2, the original, unmodified project02.apk file was executed in the Android simulator. By entering the hardcoded username (**user457**) and the code just retrieved from Ghidra (**UNIPI{My_53cuR3_p@sS!}**), the application allowed entry, displaying the success message.



In accordance with the requirements of the exercise in README.txt file, it is requested to record three different approaches to bypass the linkage mechanism. Having already successfully completed Smali Patching and Native Reverse Engineering, the possibility of an alternative static method, such as Activity/Intent Manipulation, was explored.

Manifest Bypass Case

A common static bypass technique involves modifying the AndroidManifest.xml file (via Apktool). If the application had a separate, protected screen (e.g. SecondActivity), it would be possible to transfer the intent filter "<category android:name=\"android.intent.category.LAUNCHER\"/>" to it, forcing the operating system to load the internal screen directly, bypassing the Login screen completely. However, upon inspection of the decompressed AndroidManifest.xml, it was found that the application implements its architecture exclusively in a single activity (tag <activity>): com.unipi.project02.MainActivity.



The absence of alternative activities technically knocks down the potential of Manifest Bypass. The app does not route the user to a new screen, but performs the check and renders the result (via Toast message) in the same environment. Based on the exclusion of this static method, it is concluded that the appropriate third approach (which is not mentioned in the tips of the exercise) goes into the field of dynamic analysis. The use of tools such as **Frida** it would allow the device's memory to be tampered with at runtime, by dynamically replacing the return value of the login() method or by stealing the code from the RAM before the comparison process.

Method 3: Dynamic analysis with Frida.

Having considered the static intervention approaches (Smali Patching and Ghidra Decompilation), the third method focuses on the pillar of dynamic analysis. In this approach, the aim is not to modify the physical executable file (APK), but to directly intervene in the memory (RAM) of the device and in the sequence of execution of instructions during the runtime of the application. For the implementation of this attack, the leading open source framework was chosen **Frida**.

A basic requirement for the start of dynamic analysis is the installation of the original (non-modified) version of the application (project02.apk) on a virtual device (Android Emulator). The Frida tool is based on a client-server architecture, which makes it necessary to host and continuously run a standalone file (frida-server) in the background of the target device's operating system. As this tool interacts directly with the memory and processes of third-party applications, it is imperative to obtain administrator/superuser (Root) rights on the Android system.

During the initial attempt to scale privileges through the Android Debug Bridge (ADB) command-line tool, using the adb root command, the system rejected the request by returning the error "adb cannot run as root in production builds". This bug highlights a key security mechanism of the Android ecosystem. The virtual device (Pixel 10 Pro XL) that was originally used was created with a System Image that

incorporated the Google Play Store. Google qualifies these images as Production Builds. Aimed at simulating a real, commercial device, the operating system in these versions is tightly locked, preventing ADB (adb) from running with root privileges. In order to circumvent this architectural limitation, it was decided to create a new virtual device through the Android Studio Device Manager.

The "key" to success in this step was the strategic selection of the appropriate software version. Instead of images with Google Play support, a system image based exclusively on **Google APIs** (processor architecture x86_64) was chosen. These versions (Target Images) are customized, as they do not carry the production restrictions and allow the free acquisition of Root privileges. Once the application was installed project02.apk on the new, open virtual appliance, the privilege escalation command was re-executed by providing the full path of the ADB executable: **C:\Users\Kalai\AppData\Local\Android\Sdk\platform-tools\adb root**. The system responded immediately with the message "restarting adbd as root", indicating that the Android Debug Bridge background service was successfully terminated and restarted as a superuser process, now providing complete control over the file system and the mobile memory.

```
C:\Users\Kalai\Desktop\Android_Project_2>C:\Users\Kalai\AppData\Local\Android\Sdk\platform-tools\adb root
restarting adbd as root
```

In order for the Frida framework to access the operating system processes, the existence of the corresponding server (frida-server) within the device is required. Based on the architecture of the created simulator (x86_64), the corresponding version of the server was downloaded from the official repository. The file was decompressed, renamed to "frida-server" for ease of management, and placed in the central working directory (Android_Project_2). The process of installing and running the server on the virtual machine was implemented in three successive stages through the terminal, leveraging the capabilities of ADB:

1. **Transfer of the executable file:** The adb push command was used to securely transfer the file from the local system (host) to the temporary directory of the virtual device (/data/local/tmp/), which is traditionally used to run external tools.

```
C:\Users\Kalai\Desktop\Android_Project_2>C:\Users\Kalai\AppData\Local\Android\Sdk\platform-tools\adb push frida-server /
data/local/tmp/
frida-server: 1 file pushed, 0 skipped, 155.5 MB/s (111475776 bytes in 0.684s)
C:\Users\Kalai\Desktop\Android_Project_2>
```

2. **Attribution of execution privileges:** Since the Android operating system is based on the Linux kernel, the transferred file did not have original execution rights. Through the **adb shell command "chmod 755 /data/local/tmp/frida-server"**, the permissions of the file were modified, allowing the user (owner) to execute it.

```
C:\Users\Walai\Desktop\Android_Project_2>C:\Users\Walai\AppData\Local\Android\Sdk\platform-tools\adb shell "chmod 755 /data/local/tmp/frida-server"
C:\Users\Walai\Desktop\Android_Project_2>
```

3. **Starting the service in the background:** To keep the server active without committing the terminal session, the **adb shell command "/data/local/tmp/frida-server &"** was executed. Adding the "&" symbol to the end of the command ensures that the process will run silently in the background. The absence of error messages and the immediate return to an empty prompt confirmed the successful start of the server.

```
C:\Users\Walai\Desktop\Android_Project_2>C:\Users\Walai\AppData\Local\Android\Sdk\platform-tools\adb shell "/data/local/tmp/frida-server &"
```

With the server in full readiness, the next stage involved writing the code that would "inject" the application's memory. For this purpose, a specialized script in JavaScript language called `bypass.js` was created. In order to demonstrate a complete understanding of the hooking mechanism, the code was designed with optimal error management and detailed logging in mind. The code developed is listed below:

```
if (Java.available) {
  Java.perform(function () {
    console.log("[+] Dynamic instrumentation environment initialized successfully.");

    Retrieve the app's class from memory
    var targetActivity = Java.use("com.unipi.project02.MainActivity");

    Bypassing the login() method by redefining its implementation
    targetActivity.login.implementation = function () {
      console.log("[+] Intercepted call to: MainActivity.login()");
      console.log("[+] Diverting execution flow to bypass hardcoded verification.");
```

```

Dynamically loading the necessary classes of the Android SDK
var androidToast = Java.use("android.widget.Toast");
var javaString = Java.use("java.lang.String");

Download the context of the current application (Application Context)
var currentContext = this.getApplicationContext();

Build and view your custom success message
var bypassNotification = javaString.$new("Login Successful! (Frida Bypass)");
androidToast.makeText(currentContext, bypassNotification, 0).show();

console.log("[+] Custom notification injected. Authentication controls
bypassed.");
    };
  });
}

```

The script first checks the availability of the Java environment (`Java.available`) and creates a secure execution thread (`Java.perform`). Through the `Java.use` function, Frida detects the already loaded `MainActivity` class in the RAM. The essence of dynamic override lies in the `.implementation` property. The original logic of the `login()` function (which contains the equals checks with the credentials) is completely replaced by the new custom function. The new function ignores the user's input data and simply leverages the Android SDK (via the `android.widget.Toast` and `java.lang.String` classes) to display the successful sign-in message directly.

With the `bypass.js` file stored in the working folder, a new terminal window opened (keeping the server active in the previous one). With the virtual device open, the order to carry out the attack was given: **`frida -U -f com.unipi.project02 -l bypass.js`**. The `"-U"` parameter defined the connection to the virtual device, the `"-f"` parameter forced the application to boot from scratch, and the `"-l"` parameter fed the intervention code.

```
Command Prompt - frida -U x + -
Microsoft Windows [Version 10.0.26290.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Kalai>cd Desktop\Android_Project_2

C:\Users\Kalai\Desktop\Android_Project_2>frida -U -f com.unipi.project02 -i bypass.js

  /-|
 | C |
  /-|

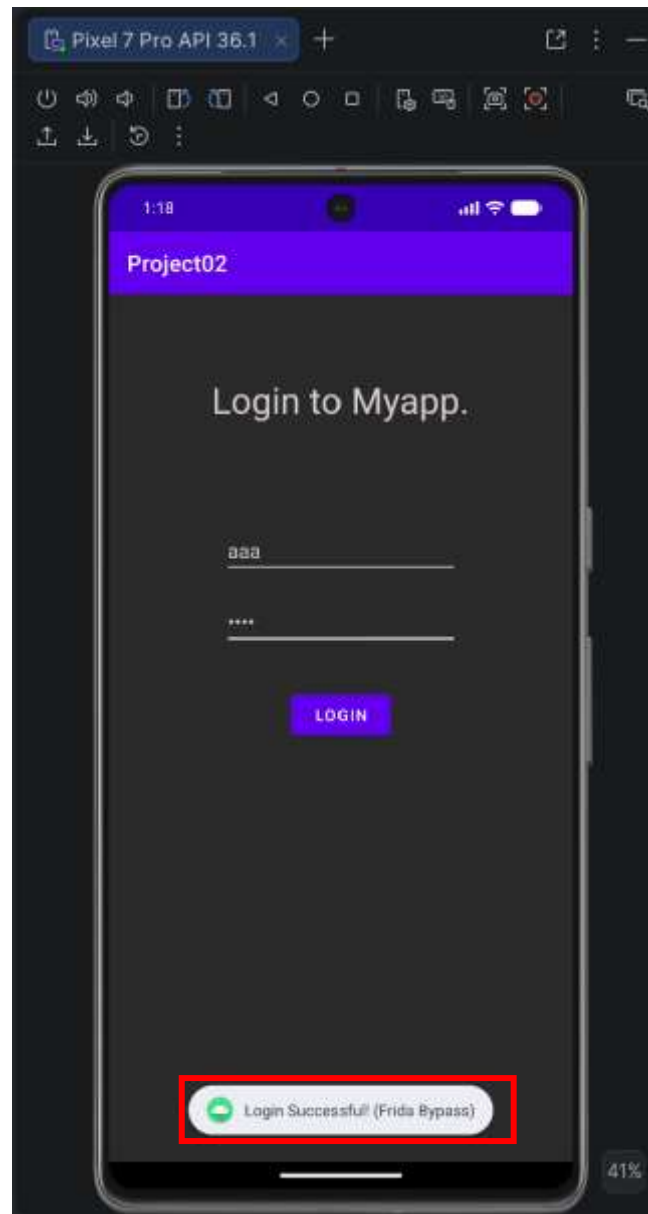
Frida 17.14.1 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to Android Emulator 5554 (id=emulator-5554)
Spawning 'com.unipi.project02'. Resuming main thread!
[Android Emulator 5554::com.unipi.project02 ]-> [*] To Frida Script φερτὴθηκε ἐπιτυχῶς!
```

Once the app was launched on the simulator screen, a test was performed by randomly entering invalid data into both the Username field (e.g. 'aaa') and the Password field (e.g. 'bbbb'). With the activation of the "LOGIN" button, the intervention code (hook) immediately prevailed. The app, having lost its original verification logic, allowed login by displaying the custom notification message: "Login Successful! (Frida Bypass)».



This behavior is the indisputable confirmation of the success of method 3, practically demonstrating how dynamic analysis can subjugate the execution flow of an application in real time, completely nullifying its local security mechanisms.

Conclusion

This exercise highlighted the complexity and multiple vulnerabilities of authentication mechanisms in the Android ecosystem. The application was attempted to protect itself by transferring the sensitive code (Flag) to an external native C/C++ (JNI) library. Despite the increased level of difficulty compared to simply storing the

previous exercise (UC1), it turned out that no protection alone is inviolable. The approach was carried out through three different pillars of Reverse Engineering:

1. **Logical Bypass (Smali Patching):** It confirmed that trust in the static flow of code is precarious, as the application was rebuilt to accept incorrect codes.
2. **Reverse Engineering Native Code (Ghidra):** It proved that machine code, even with cloaking techniques like stack strings, can be completely deciphered by revealing the coveted Flag.
3. **Dynamic Analysis (Frida):** Demonstrated the power of runtime interference, where the application's logic can be tampered with, without tampering with the APK file.

Overall, the paper confirms the necessity of multiple layers of security, such as using server-side validation and strong encryption, rather than simply hiding credentials in the device's code.

Bibliography

- Releases · skylot/jadx. (n.d.). Retrieved from <https://github.com/skylot/jadx/releases>

- Releases · patrickfav/uber-apk-signer. (n.d.). Retrieved from <https://github.com/patrickfav/uber-apk-signer/releases>
- Releases · NationalSecurityAgency/ghidra. (n.d.). Retrieved from <https://github.com/NationalSecurityAgency/ghidra/releases>
- Releases · frida/frida. (n.d.). Retrieved from <https://github.com/frida/frida/releases>
- OWASP MASTG - OWASP Mobile Application Security. (n.d.). Retrieved from <https://mas.owasp.org/MASTG/>
- ApkTool | ApkTool. (n.d.). Retrieved from <https://ibotpeaches.github.io/Apktool/>
- Frida • A world-class dynamic instrumentation toolkit. (n.d.). Retrieved from <https://frida.re/>